

Markdown Studio 보안 검토 보고서

본 문서는 검토 범위와 발견 사항을 정리합니다. 각 항목의 구체적 수정 방법·회귀 테스트 절차는 [[MAINTENANCE]]를 참조하십시오.

1. 개요

항목	내용
검토 대상	markdown-studio.html (단일 파일, 의존성 없음) + desktop/ (Electron 래퍼·패키징)
검토 일	2026-08-04
검토 범위	마크다운 렌더러, 링크·이미지 처리, 원시 HTML 통과 경로, HTML/PDF/DOCX 내보내기, 파일 입출력 (FSA-IndexedDB), 드래그앤드롭, Electron 메인 프로세스(창·네비게이션·권한·프로토콜), 배포(패키징·서명·Smart App Control)
검토 방법	소스 정독 + 브라우저 실행 기반 XSS/CSP 검증(Chrome) + Electron 셸프테스트(개발·패키지·포터블 exe)
검토 결과	검토 항목 전부 조치 완료 — 현재 알려진 보안 이슈 없음

결론: 현재 프로그램에는 알려진 보안 이슈가 없습니다. 검토 항목은 모두 조치되어 브라우저·Electron 검증을 통과했습니다. 상세는 §6-§8.

위협 모델

(A) 웹 앱 — 핵심 신뢰 경계는 "열어들이는 .md 파일의 내용"입니다.

사용자는 공유·다운로드받거나 타인이 작성한 마크다운 파일을 엽니다. 그 내용은 ①미리보기에서 앱과 같은 출처(origin)로 렌더링되고, ②HTML로 내보내면 페이로드가 산출물에 실려 2차 전파되며, ③앱은 File System Access API로 로컬 파일을 덮어쓸 권한을 가집니다. 따라서 문서 본문에서 임의 스크립트가 실행되면(스토어드 XSS) 열람자 컨텍스트 탈취·파일 조작·피해 확산으로 이어집니다. **최우선 검토 대상은 렌더러의 XSS입니다.**

(B) 데스크톱 — Electron 특권 컨텍스트와 배포 무결성.

데스크톱 빌드는 FSA·IndexedDB를 쓰기 위해 HTML을 특권 스킴 `app://` (standard + secure)로 서빙합니다. 이 특권 창이 외부 콘텐츠로 이동하거나 새 창을 무제한 생성하면 위험이 커집니다. 또한 배포 산출물(exe)의 무결성·신뢰(코드 서명/OS의 앱 제어) 역시 검토 대상입니다.

2. 검토한 항목

- **마크다운 렌더러** — 인라인/블록 원시 HTML 통과 경로, 링크·이미지 URL 처리, 프론트매터, 코드 블록.
 - **내보내기** — HTML(자체완결형)·PDF(iframe 인쇄)·DOCX(수작업 OOXML) 생성 경로와 그 안의 문자열/속성 이스케이프.
 - **심층 방어** — 콘텐츠 보안 정책(CSP) 유무.
 - **파일 입출력** — File System Access API, IndexedDB(최근 파일), 드래그앤드롭, 상대경로 이미지 해석.
 - **데스크톱(Electron)** — `app://` 스킴, 창 생성/네비게이션 정책, 렌더러 격리·샌드박스·권한, 파일 연결.
 - **배포** — 패키징 산출물의 코드 서명·무결성, Windows Smart App Control 동작.
-

3. 발견 사항 요약

#	발견	위치	심각도	상태
V1	인라인 원시 HTML이 살균 없이 미리보기에 삽입 (이벤트 핸들러 XSS)	inline()	치명	✅ 해결
V2	블록 원시 HTML(<iframe> · <svg onload> 등)이 살균 없이 삽입	render()	치명	✅ 해결
V3	링크 URL 스킴 미검증(javascript: 등 실행 가능)	inline()	높음	✅ 해결
V4	이미지 URL 스킴 미검증(data:image/svg 등)	inline()	높음	✅ 해결
V5	DOCX 관계(Target) 속성 인젝션(따옴표 미이스케이프)	docxEsc()	중간	✅ 해결
V6	DOCX 하이퍼링크에 javascript: 스킴 통과	buildDocxBlob()	중간	✅ 해결
D1	Electron 새 창 생성·외부 네비게이션 무제한	main.js	중간	✅ 해결
V7	CSP(콘텐츠 보안 정책) 부재 — 심층 방어 없음	<head>	낮음	✅ 해결
D2	Electron 렌더러 샌드박스·권한 제한 부재	main.js	낮음	✅ 해결
V8	내보내기 시 임의 URL 이미지 fetch(경미한 정보 노출)	inlineImages()	낮음	✅ 해결(설계상 수용, §7)

배포 통제 이슈: 패키징 exe가 Windows Smart App Control에 차단 → **해결 완료(\$6)**.

4. 발견 사항 상세 (검토 내용)

각 항목은 무엇이 문제였는지(원인·재현·영향) 중심으로 기술합니다. 모두 조치 완료 상태입니다.

4.1 렌더러 XSS

V1 — 인라인 원시 HTML 무살균 삽입 (치명)

렌더러가 인라인 HTML 태그를 자리표시자로 가려 두었다가 **아무 필터 없이 그대로**

preview.innerHTML 에 되돌려 넣었습니다. 안녕 하세요 한 줄로 이미지 로드 실패 시 임의 JS가 실행됩니다. 임의 스크립트 실행은 세션 탈취·로컬 파일 조작·내보낸 HTML 로의 전파로 이어집니다. —  해결

V2 — 블록 원시 HTML 무살균 삽입 (치명)

블록 레벨 원시 HTML 처리가 위험 태그(iframe · svg · script · style · form)를 포함해 그대로 출력했습니다. <svg onload="...">, <iframe src="javascript:..."> 등이 innerHTML 에서 실행됩니다. V1과 동일한 치명적 XSS. —  해결

V3 — 링크 URL 스킴 미검증 (높음)

[텍스트](URL) 로 앵커를 만들 때 스킴을 검증하지 않아 [클릭](javascript:...) 이 그대로 href 에 들어갔고, 엔터티로 위장한 콜론(javascript:...)도 브라우저 디코딩 후 실행될 수 있었습니다. —  해결

V4 — 이미지 URL 스킴 미검증 (높음)

![alt](src) 의 src 를 검증하지 않아 data:image/svg+xml 등 스크립트 삽입 통로가 될 수 있는 값이 통과했습니다. —  해결

4.2 내보내기(DOCX)

V5 — DOCX 관계 Target 속성 인젝션 (중간)

DOCX 문자열 이스케이프가 & · < · > 만 처리하고 따옴표를 두어, XML 속성값(Target="URL")에 따옴표가 든 URL이 들어가면 관계(relationship) XML이 깨지거나 조작될 수 있었습니다. —  해결

V6 — DOCX 하이퍼링크 javascript: 통과 (중간)

DOCX 하이퍼링크 대상 URL의 스킴을 검증하지 않았습니다. —  해결

4.3 심층 방어(CSP)

V7 — CSP 부재 (낮음)

앱 본체와 내보낸 HTML 모두 Content-Security-Policy 가 없어, 살균기 외 2차 방어선(스크립트/eval/ <object> / <base> /폼 전송/프레임 제한)이 없었습니다. —  해결

4.4 데스크톱(Electron)

D1 — 새 창·외부 네비게이션 무제한 (중간)

새 창(BrowserWindow) 생성을 임의 스킴에 허용했고, 특권 app:// 창이 외부로 top-level 이동하

는 것을 막는 방어가 없었습니다. 악성 문서의 리다이렉트 유도 시 특권 창에 외부 콘텐츠가 실릴 여지가 있었습니다. —  **해결**

D2 — 렌더러 샌드박스-권한 제한 부재 (낮음)

렌더러 샌드박스가 명시되지 않았고, 위치·카메라·마이크·알림 등 권한 요청을 제한하는 정책이 없었습니다. —  **해결** (샌드박스 적용 후에도 FSA-secure context 정상 동작 확인.)

5. (배포) Smart App Control 차단

증상: 패키징한 Markdown Studio.exe 실행 시 Windows 11 Smart App Control이 "안전하지 않을 수 있는 앱"으로 차단.

원인: 패키저가 원본 electron.exe (Microsoft ISG 클라우드 평판 보유)를 재기록하며 바이트를 바꿔 평판 해시가 소멸.

상태:  **해결** — 원본 바이너리를 바이트 그대로 유지하는 포터블 빌드로 평판을 보존해 SAC를 통과. 원본과 SHA-256 해시 일치 및 실제 exe 셀프테스트로 검증. (상세·정식 서명 방안은 [[MAINTENANCE]] 및 desktop/README.md.)

6. 검증 결과

6.1 브라우저 XSS(살아있는 DOM 삽입 + 이벤트 강제 디스패치)

페이로드	결과
	미발화
<svg onload=...> / <iframe src="javascript:..."> / <script>...	제거
[x](javascript:...) / ![x](javascript:...)	앵커·이미지 미생성(평문 노출)
[x](javascript:...) (엔터티 위장)	차단
 / <div onmouseover=...>	클릭·호버 강제에도 미발화
정상: · <kbd> · <sub> · https:// 링크·이미지	모두 보존

→ 실행된 스크립트 0건, 정상 서식 보존, 콘솔 오류 없음.

6.2 CSP 회귀

항목	결과
로드 시 CSP 위반(Refused to...)	없음
미리보기·HTML 내보내기·PDF용 iframe(인라인 style)	정상

6.3 Electron 셀프테스트(개발·패키지·포터블 exe)

검사	결과
편집기 DOM 부팅 / 마크다운 렌더	PASS
보안 컨텍스트·FSA — sandbox 하에서	PASS
파일 인자 오픈(읽기전용)	PASS (docs=1 [*.md(RO)])
포터블 exe SAC 차단	없음

7. 설계상 수용 사항 (참고)

V8 — 내보내기 시 이미지 fetch: HTML/PDF/DOCX로 내보낼 때 문서의 이미지를 base64로 내장하기 위해 이미지 URL을 fetch 합니다. 악의적 문서가 내부망 주소를 가리키면 열람자 브라우저가 그 주소를 조회할 수 있습니다(경미). **이미지 내장이라는 정상 기능에 내재된 동작**이라 별도 결함으로 보지 않고 수용하며, 필요 시 사설/내부망 주소 차단 옵션으로 완화할 수 있습니다.

8. 결론

검토한 모든 항목이 조치되어 **현재 프로그램에는 알려진 보안 이슈가 없습니다**. 웹 앱은 렌더러 살균 + 이중 CSP로, 데스크톱은 Electron 하드닝으로 위협 모델의 주요 공격면을 차단했으며, 브라우저·Electron 실행 기반 검증(실행된 스크립트 0건, CSP 위반 0건, 셀프테스트 전 항목 PASS)을 통과했습니다.

후속 강화 권고(코드 서명·DOMPurify 등)와 회귀 테스트 절차는 [[MAINTENANCE]]에 정리되어 있습니다.